

Dialogue Systems

Frame-based Dialogue-state Architecture

Dr. Saeedeh Momtazi



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)



آکادمی‌ها



- **The Dialogue-State Architecture**
- Dialogue Acts
- Slot Filling
- Dialogue State Tracking
- Dialogue Policy
- NLG in the Dialogue-state Model

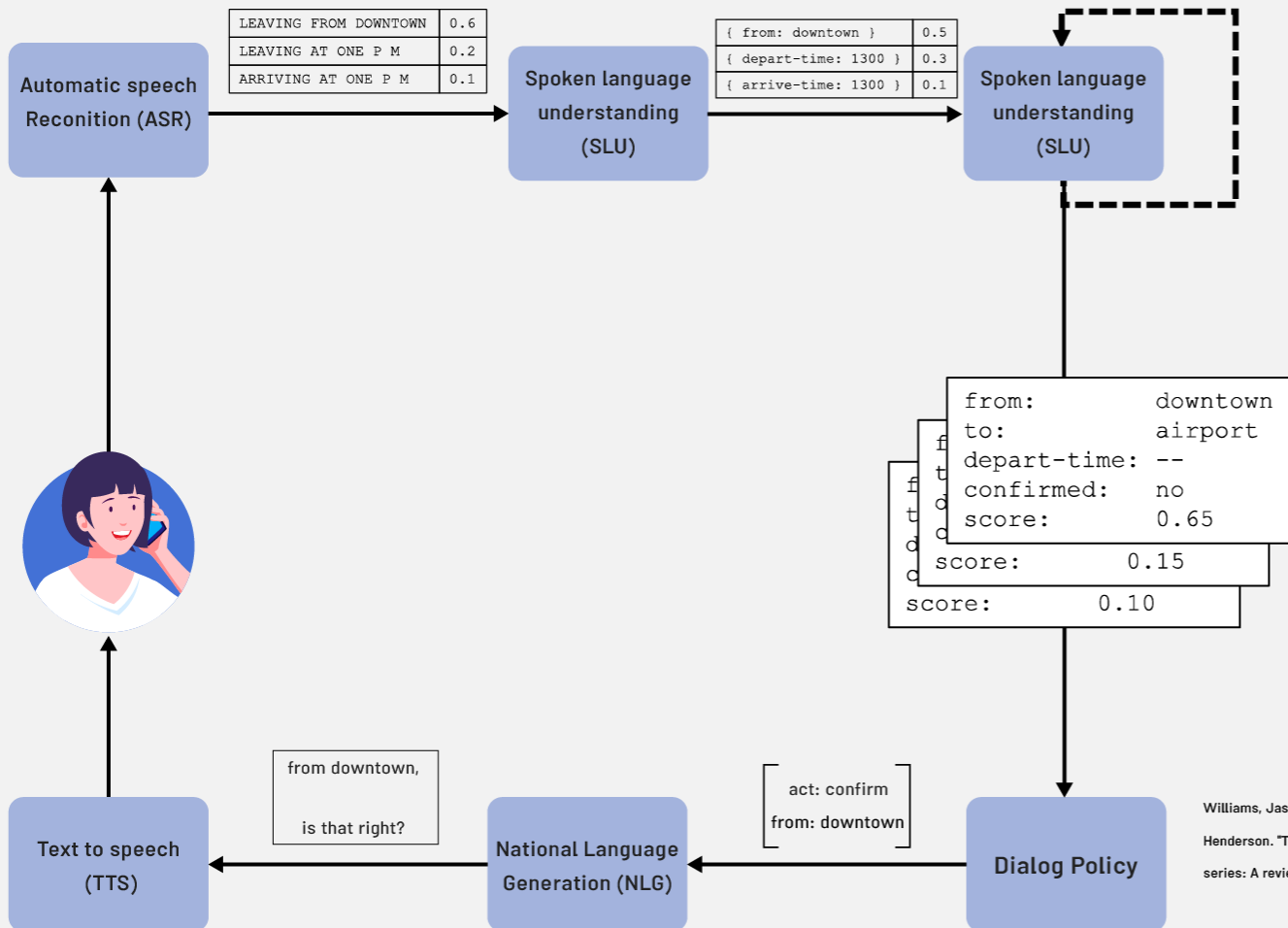


Dialogue-State or Belief-State Architecture

- A more sophisticated version of the frame-based architecture
 - ▶ Has dialogue acts, more ML, better generation
- The basis for modern research systems
- Slowly making its way into industrial systems
 - ▶ Some aspects (ML for slot-understanding) already widely used industrially



The Dialogue-State Architecture



Williams, Jason D., Antoine Raux, and Matthew Henderson. "The dialog state tracking challenge series: A review." *Dialogue & Discourse* 7, no. 3 (2016): 4-33.



Components in a Dialogue-State Architecture

- **NLU:** extracts slot fillers from the user's utterance using machine learning
- **Dialogue state tracker:** maintains the current state of the dialogue (user's most recent dialogue act, set of slot-filler constraints from user)
- **Dialogue policy:** decides what the system should do or say next
 - ▶ GUS policy: ask questions until the frame was full then report back
 - ▶ More sophisticated: know when to answer questions, when to ask a clarification question, etc
- **NLG:** produce more natural, less templated utterances



Dialogue-State Architecture vs GUS

- Slot fillers use ML rather than rules
- Dialogue policy can help a system decide
 - ▶ When to answer the user's questions
 - ▶ When to instead ask the user a clarification question
 - ▶ When to make a suggestion
 - ...
- Dialogue state systems have a natural language generation component.
 - ▶ In GUS, the sentences that the generator produced were all from pre-written templates
 - ▶ But a more sophisticated generation component can condition on the exact context to produce turns that seem much more natural



- The Dialogue-State Architecture
- **Dialogue Acts**
- Slot Filling
- Dialogue State Tracking
- Dialogue Policy
- NLG in the Dialogue-state Model



Dialogue Acts

- Dialogue acts represent the interactive function of the turn or sentence
- Combine the ideas of speech acts and grounding into a single representation
- Different types of dialogue systems require labeling different kinds of acts
 - ▶ The tagset —defining what a dialogue act is exactly— tends to be designed for particular tasks



➤ Sample Tagset

Tag	Sys	User	Description
HELLO($a = x, b = y, \dots$)	✓	✓	Open a dialogue and give info $a = x, b = y, \dots$
INFORM($a = x, b = y, \dots$)	✓	✓	Give info $a = x, b = y, \dots$
REQUEST($a, b = x, \dots$)	✓	✓	Request value for a given $b = x, \dots$
REQALTS($a = x, \dots$)	✗	✓	Request alternative with $a = x, \dots$
CONFIRM($a = x, b = y, \dots$)	✓	✓	Explicitly confirm $a = x, b = y, \dots$
CONFREQ($a = x, \dots, d$)	✓	✗	Implicitly confirm $a = x, \dots$ and request value of d
SELECT($a = x, a = y$)	✓	✗	Implicitly confirm $a = x, \dots$ and request value of a
AFFIRM($a = x, b = y, \dots$)	✓	✓	Affirm and give further info $a = x, b = y, \dots$
NEGATE($a = x$)	✗	✓	Negate and give corrected value $a = x$
DENY($a = x$)	✗	✓	Deny that $a = x$
BYE()	✓	✓	Close a dialogue

(Young et al., 2010)



Dialogue Acts

➤ Sample data (The content of each dialogue acts communicates with the slot fillers)

Utterance	Dialogue act
U: Hi, I am looking for somewhere to eat.	hello(task = find,type=restaurant)
S: You are looking for a restaurant. What type of food do you like?	confreq(type = restaurant, food)
U: I'd like an Italian somewhere near the museum.	inform(food = Italian, near=museum)
S: Roma is a nice Italian restaurant near the museum.	inform(name = "Roma", type = restaurant, food = Italian, near = museum)
U: Is it reasonably priced?	confirm(pricerange = moderate)
S: Yes, Roma is in the moderate price range.	affirm(name = "Roma", pricerange = moderate)
U: What is the phone number?	request(phone)
S: The number of Roma is 385456.	inform(name = "Roma", phone = "385456")
U: Ok, thank you goodbye.	bye()

(Young et al., 2010)



- The Dialogue-State Architecture
- Dialogue Acts
- **Slot Filling**
- Dialogue State Tracking
- Dialogue Policy
- NLG in the Dialogue-state Model



Slot Filling: Machine learning

- Machine learning classifiers to map words to semantic frame-fillers
- Given a set of labeled sentences

Input: "I want to fly to San Francisco on Monday afternoon please"

Output: Destination: SF

Depart-time: Monday afternoon

- Build a classifier to map from one to the other
- Requirements: Lots of labeled data



Slot Filling as Sequence Labeling

➤ The **BIO tagging** paradigm

➤ Idea: Train a classifier to label each input word with a tag that tells us what slot (if any) it fills

0	0	0	0	0	B-DES	I-DES	0	B-DEPTIME	I-DEPTIME	0
I	want	to	fly	to	San	Francisco	on	Monday	afternoon	please

➤ We create a B and I tag for each slot-type

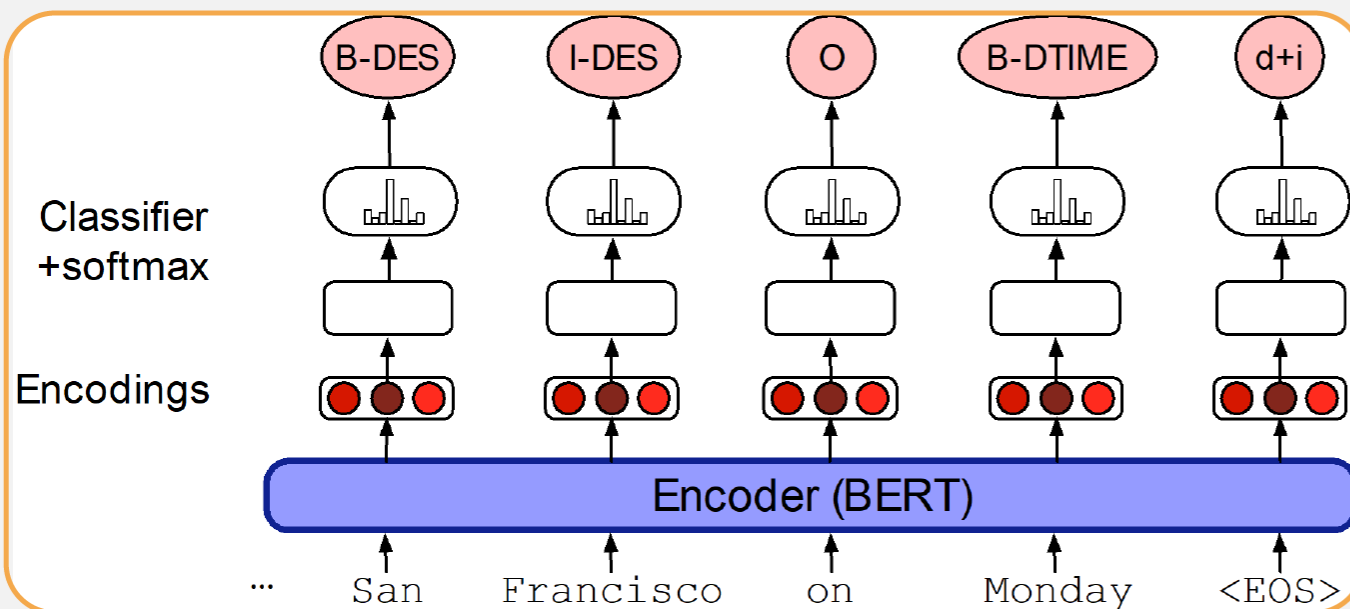
➤ And convert the training data to this format



Slot Filling Using Contextual Embeddings

➤ Can do domain and intent too:

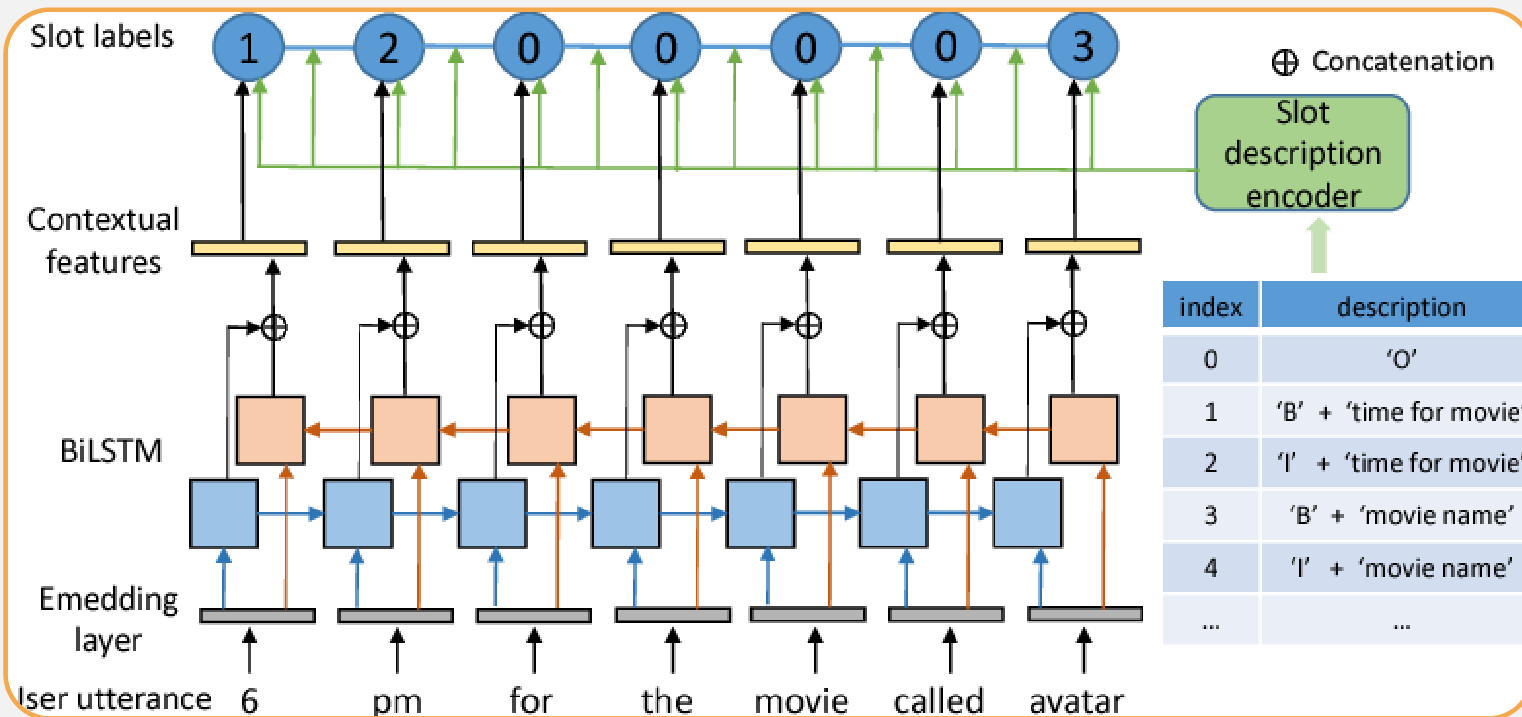
▶ e.g., generate the label "AIRLINE_TRAVEL + SEARCH_FLIGHT"





Slot Filling Using RNNs

➤ Using BiLSTM





Once we have the
BIO tag of the sentence

0	0	0	0	0	B-DES	I-DES	0	B-DEPTIME	I-DEPTIME	0
I	want	to	fly	to	San	Francisco	on	Monday	afternoon	please

- We can extract the filler string for each slot
- And then normalize it to the correct form in the ontology
- Like "SFO" for San Francisco
- Using homonym dictionaries (SF=SFO=San Francisco)



- The Dialogue-State Architecture
- Dialogue Acts
- Slot Filling
- **Dialogue State Tracking**
- Dialogue Policy
- NLG in the Dialogue-state Model



Dialogue State Tracking

- The job of the dialogue-state tracker is to determine
 - ▶ The current state of the frame (the fillers of each slot)
 - ▶ As well as the user's most recent dialogue act
- Dialogue-state includes more than just the slot-fillers expressed in the current sentence
- It includes the entire state of the frame at this point, summarizing all of the user's constraints



Dialogue State Tracking

User: I'm looking for a cheaper restaurant

`inform(price=cheap)`

System: Sure. What kind - and where?

User: Thai food, somewhere downtown

`inform(price=cheap, food=Thai, area=centre)`

System: The House serves cheap Thai food

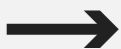
User: Where is it?

`inform(price=cheap, food=Thai, area=centre); request(address)`

System: The House is at 106 Regent Street



➤ I'd like Cantonese food near the Mission district.



inform(food=cantonese, area=mission).

➤ Dialogue act interpretation use a supervised classification algorithm to choose inform

▶ Used handlabeled dialog acts to predicting the dialogue act tag

▶ Based on Encodings of current sentence + prior dialogue acts



Simple and Complex Dialogue State Tracking

➤ Simple dialogue state tracker:

- ▶ Run a slot-filler after each sentence

➤ More complex model:

- ▶ Can make use of the reading-comprehension architectures
- ▶ e.g., the model of Gao et al. (2019) trains a classifier for each slot to decide whether its value is being changed in the current sentence or should be carried over from the previous sentences
- ▶ If the slot value is being changed, a span-prediction model is used to predict the start and end of the span with the slot filler



Detecting Correction Acts

- An special case of dialogue act detection
- If system misrecognizes an utterance
- User might make a **correction**
 - ▶ Repeat themselves
 - ▶ Rephrasing
 - ▶ Saying "no" to a confirmation question



Detecting Correction Acts

- Corrections are harder to recognize!
- From speech, corrections are misrecognized twice as often (in terms of word error rate) as non-corrections! (Swerts et al 2000)
- Hyperarticulation (exaggerated prosody) is a large factor:
 - ▶ "I said BAL-TI-MORE, not Boston"



Detecting Correction Acts

- User corrections tend to be either exact repetitions or repetitions with one or more words omitted, although they may also be paraphrases of the original utterance
- Detecting reformulations or correction acts can be part of the general dialogue act detection classifier
- Alternatively, because the cues to these acts tend to appear in different ways than for simple acts (like INFORM or REQUEST), we can make use of features orthogonal to simple contextual embedding features



Features for Detecting Corrections in Spoken Dialogue

➤ Some typical features

features	examples
lexical	words like “no”, “correction”, “I don’t”, swear words, utterance length
semantic	similarity (word overlap or embedding dot product) between the candidate correction act and the user’s prior utterance
phonetic	phonetic overlap between the candidate correction act and the user’s prior utterance (i.e. “WhatsApp” may be incorrectly recognized as “What’s up”)
prosodic	hyperarticulation, increases in F0 range, pause duration, and word duration, generally normalized by the values for previous sentences
ASR	ASR confidence, language model probability



- The Dialogue-State Architecture
- Dialogue Acts
- Slot Filling
- Dialogue State Tracking
- **Dialogue Policy**
- NLG in the Dialogue-state Model



Dialogue Policy

➤ At turn i predict action A_i to take, given entire history:

$$\hat{A}_i = \operatorname{argmax}_{A_i \in A} P(A_i | (A_1, U_1, \dots, A_{i-1}, U_{i-1}))$$

➤ Simplify by just conditioning on

- ▶ The current dialogue state (filled frame slots)
- ▶ The last turn by system
- ▶ The last turn by user

$$\hat{A}_i = \operatorname{argmax}_{A_i \in A} P(A_i | \text{Frame}_{i-1}, A_{i-1}, U_{i-1})$$



Dialogue Policy with Reinforcement Learning

- To decide which action to take, a reinforcement learning system gets a reward at the end of the dialogue, and uses that reward to train a policy to take actions
- Example: a movie-recommendation dialogue system (Fazel-Zarandi et al. 2017)
 - The action space has only three actions: EXECUTE, CONFIRM, and ELICIT
 - **EXECUTE** which sends a query to the database and answers the user's question
 - **CONFIRM** which clarifies the intent or slot with the users (e.g., "Do you want movies directed by Christopher Nolan?")
 - **ELICIT** which asks the user for missing information (e.g., "Which movie are you talking about?")
 - The system rewards includes:
 - A large positive reward if the dialogue system terminates with the correct slot representation at the end
 - A large negative reward if the slots are wrong
 - A small negative reward for confirmation and elicitation questions to keep the system from re-confirming everything



Policy Example: Confirmation and Rejection

- Dialogue systems make errors
- So they to make sure they have understood user
- Two important mechanisms:
 - ▶ **Confirming** understandings with the user
 - ▶ **Rejecting** utterances that the system is likely to have misunderstood



Explicit Confirmation Strategy

S: Which city do you want to leave from?

U: Baltimore.

S: **Do you want to leave from Baltimore?**

U: Yes.

U: I'd like to fly from Denver Colorado to New York City on September twenty first in the morning on United Airlines

S: **Let's see then. I have you going from Denver Colorado to New York on September twenty first. Is that correct?**

U: Yes



Implicit Confirmation Strategy

U: I want to travel to Berlin

S: **When do you want to travel to Berlin?**

U2: Hi I'd like to fly to Seattle Tuesday Morning

A3: **Traveling to Seattle on Tuesday, August eleventh in the morning.**
Your full name?



Confirmation Strategy Tradeoffs

- Explicit confirmation makes it easier for users to correct the system's misrecognitions since a user can just answer "no" to the confirmation question
- But explicit confirmation is also awkward and increases the length of the conversation (Danieli and Gerbino 1995, Walker et al. 1998)
- The explicit confirmation dialogue fragments above sound non-natural and definitely non-human
- Implicit confirmation is much more conversationally natural



I'm sorry, I didn't understand that.



Progressive Prompting for Rejection

- Also known as escalating detail
- Give user guidance about what they can say
 - ▶ Don't just repeat the question "When would you like to leave?"

System: When would you like to leave?

Caller: Well, um, I need to be in New York in time for the first World Series game.

System: <reject>. Sorry, I didn't get that. Please say the month and day you'd like to leave.

Caller: I wanna go on October fifteenth.



Rapid Reprompting

- The system rejects an utterance just by saying
 - ▶ "I'm sorry?"
 - ▶ "What was that?"
- Only if the caller's utterance is rejected a second time, the system starts applying progressive prompting
- Experiments showed that users greatly prefer rapid reprompting as a first-level error prompt (Cohen et al., 2004)



Using Confidence to Decide Whether to Confirm

➤ ASR or NLU systems can assign a confidence value, indicating how likely they are that they understood the user.

- ▶ Acoustic log-likelihood of the utterance
- ▶ Prosodic features
- ▶ Ratio of score of best to second-best interpretation

➤ Systems could use set confidence thresholds:

$< \alpha$	low confidence	reject
$\geq \alpha$	above the threshold	confirm explicitly
$\geq \beta$	high confidence	confirm implicitly
$\geq \gamma$	very high confidence	don't confirm at all



- The Dialogue-State Architecture
- Dialogue Acts
- Slot Filling
- Dialogue State Tracking
- Dialogue Policy
- **NLG in the Dialogue-state Model**



Natural Language Generation

- NLG in information-state architecture modeled in two stages:
 - ▶ **Content planning** (what to say)
 - ▶ **Sentence realization** (how to say it)
- Content planning is normally done by the dialogue policy
- We'll focus on sentence realization here



Sentence Realization

- Assume the dialogue policy performed content planning by choosing
 - ▶ The dialogue act to generate
 - ▶ Some attributes (slots and values) that the planner wants to say to the user
(Either to give the user the answer, or as part of a confirmation strategy)



Sentence Realization

➤ 2 examples

```
recommend(restaurant name= Au Midi, neighborhood = midtown,  
cuisine = french
```

- 1 Au Midi is in Midtown and serves French food.
- 2 There is a French restaurant in Midtown called Au Midi.

```
recommend(restaurant name= Loch Fyne, neighborhood = city  
centre, cuisine = seafood)
```

- 3 Loch Fyne is in the City Center and serves seafood food.
- 4 There is a seafood restaurant in the City Centre called Loch Fyne.



Sentence Realization

➤ Training data is hard to come by

▶ Don't see each restaurant in each situation

➤ Common way to improve generalization:

▶ **Delexicalization:** replacing words in the training set that represent slot values with a generic placeholder token

```
recommend(restaurant name= Au Midi, neighborhood = midtown,  
cuisine = french
```

- 1 Au Midi is in Midtown and serves French food.
- 2 There is a French restaurant in Midtown called Au Midi.

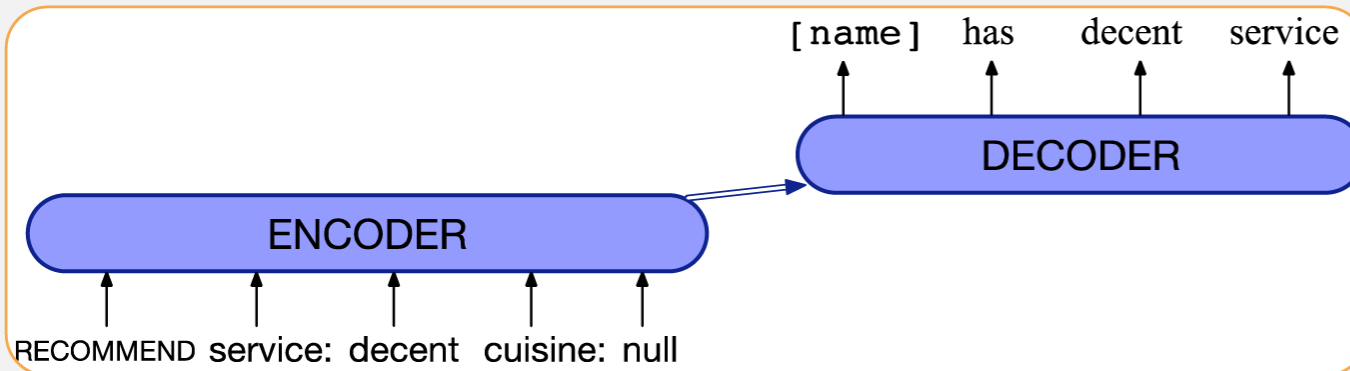
```
recommend(restaurant name= Au Midi, neighborhood = midtown,  
cuisine = french
```

- 1 restaurant_name is in neighborhood and serves cuisine food.
- 2 There is a cuisine restaurant in neighborhood called restaurant_name.



Sentence Realization: Mapping from Frames to Delexicalized Sentences

➤ Encoder-decoder models:



➤ Output: restaurant_name has decent service

➤ Relexicalize to: Au Midi has decent service



NLG for Specific Dialogue Acts

- Designing NLG algorithms that are specific to a particular dialogue act
 - ▶ e.g., generating clarification questions
- In cases where the speech recognition fails to understand some part of the user's utterance
 - ▶ Instead of generating a generic dialogue act REJECT use targeted clarification questions that reprise elements of the misunderstanding
 - ▶ ("Please repeat", or "I don't understand what you said")



Generating Clarification Questions

User: What do you have going to UNKNOWN WORD on the 5th?

System: Going where on the 5th?

- The system repeats "going" and "on the 5th" to make it clear which aspect of the user's turn the system needs to be clarified
- Methods for generating clarification questions:
 - ▶ Rules like 'replace "going to UNKNOWN WORD" with "going where"'
 - ▶ Classifiers that guess which slots were misrecognized



Further Reading

➤ Speech and Language Processing (3rd ed. draft)

▶ Chapter 24